

Journal de bord IA

J'ai utilisé deux outils d'IA générative à chaque étape clé du projet : **Claude (Sonnet 5)** pour l'ensemble de l'architecture, du code et de la documentation, et **Gemini** pour les tests unitaires du module `storage.py` ainsi que pour un test comparatif de formulations. Pour chaque étape, j'ai noté l'outil utilisé, le prompt, ma propre évaluation de la qualité de la réponse, et la correction que j'ai apportée quand c'était nécessaire.

Note méthodologique : les prompts ci-dessous sont reconstruits a posteriori à partir de mes notes de travail, sous la forme sous laquelle je les ai formulés aux outils IA. Ils reflètent fidèlement le contenu et l'intention de mes échanges, sans être nécessairement le texte caractère pour caractère tapé sur le moment.

Légende qualité

- **Correcte** : utilisable telle quelle
- **Partielle** : utile mais nécessitant une correction
- **Hallucinée** : information fausse ou inventée

Étapes clés

#	Étape	Outil	Prompt	Qualité	Correction apportée
1	Cadrage du projet	Claude Sonnet 5	« On part de zéro sur un exercice noté : réalisation d'un tableau de bord de données publiques, back-end + front-end. Propose-moi un domaine de données, une stack technique et une architecture, en justifiant tes choix. »	Correcte	Aucune ; j'ai validé les choix proposés (Open-Meteo car sans clé d'API, FastAPI + SQLite pour rester léger et sans serveur externe)
2	Vérification du format de l'API publique	Claude Sonnet 5, recherche web	« Avant de coder le module de collecte, vérifie sur le web le format exact de l'API Open-Meteo Air Quality : URL, paramètres disponibles (hourly, past_days), format de réponse JSON. »	Correcte	J'ai fait confirmer l'URL exacte (<code>air-quality-api.open-meteo.com/v1/air-quality</code>) et les paramètres (<code>hourly</code> , <code>past_days</code>) par une recherche web plutôt que de faire confiance à la mémoire du modèle sur un détail d'API qui évolue dans le temps
3	Génération du module de collecte (<code>collector.py</code>)	Claude Sonnet 5	« Écris <code>collector.py</code> : une fonction qui interroge l'API Open-Meteo pour une liste de villes. Si une ville échoue	Correcte	Aucune correction de fond ; j'ai fait valider immédiatement par des tests unitaires avec mocks

#	Étape	Outil	Prompt	Qualité	Correction apportée
			(erreur réseau, timeout), les autres villes doivent quand même être collectées. Gère les erreurs explicitement. »		
4	Génération du module de nettoyage (cleaner.py)	Claude Sonnet 5	« Écris cleaner.py : normalise les données collectées, remplace par None les valeurs hors des bornes physiques définies dans config.py, supprime les doublons (même ville + même timestamp), trie par date. »	Correcte	Aucune ; en explicitant les bornes dans config.py avant génération, j'ai évité l'ambiguïté typique sur « que faire d'une valeur aberrante »
5	Création de la structure de dossiers (bash)	Claude Sonnet 5	« Crée la structure de dossiers du projet en bash : backend/app, backend/tests, frontend, docs. »	Partielle	J'ai constaté que l'expansion d'accolades bash avait échoué silencieusement dans l'environnement d'exécution (un dossier littéral {backend avait été créé), ce qui provoquait une erreur en aval (cd impossible). J'ai fait corriger en remplaçant par des appels mkdir -p explicites, un par dossier.
6	Premier test de collecte réelle	Claude Sonnet 5	« Lance le pipeline complet avec python -m app.ingest et dis-moi si les données sont bien collectées, nettoyées et stockées. »	Partielle	L'exécution a échoué (HTTP 403) car l'environnement de développement restreint les appels sortants à une liste blanche de domaines n'incluant pas open-meteo.com . Ce n'est pas une erreur de code ; je l'ai fait documenter dans le README, et j'ai fait ajouter un script seed_demo_data.py pour pouvoir démontrer le dashboard hors-ligne via le même pipeline de nettoyage.
7	Dashboard front-end (HTML/ JS/Chart.js)	Claude Sonnet 5	« Crée un dashboard HTML/ JS avec Chart.js : au moins deux visualisations	Correcte	Quelques ajustements mineurs de style pour la

#	Étape	Outil	Prompt	Qualité	Correction apportée
			interactives (courbe temporelle + classement par ville), avec un filtre ville, un filtre polluant, et des indicateurs clés. »		cohérence visuelle (palette, typographie)
8	Génération des tests unitaires de <code>storage.py</code>	Gemini	« Écris des tests unitaires pytest pour ce module. » (envoyé avec le seul fichier <code>storage.py</code> , pas le reste du projet)	Partielle	En exécutant les tests, j'ai détecté trois problèmes : (1) un import placeholder jamais remplacé (<code>from ton_module import (...)</code> au lieu de <code>from app.storage import (...)</code>) ; (2) une fixture utilisant <code>db_path = ":memory:"</code> , qui échoue car chaque fonction de <code>storage.py</code> ouvre une connexion SQLite indépendante via <code>get_connection()</code> — en mémoire, chaque connexion recrée une base vide, donc la table créée par <code>init_db()</code> disparaît avant le premier <code>save_rows()</code> (no such table) ; j'ai corrigé en utilisant un fichier temporaire (<code>tempfile.mkstemp()</code>) partagé entre les appels ; (3) une assertion cassée et toujours vraie dans <code>test_get_latest_by_city</code> (<code>assert ... == "2026-07-09:00:00" if ... else True</code> , avec une date au format invalide), que j'ai remplacée par une assertion réelle sur la valeur attendue.
8bis	Application des 3 corrections et validation finale	Claude Sonnet 5	« Les tests de <code>storage.py</code> générés par Gemini ont 3 bugs : un import cassé, une fixture <code>:memory:</code> incompatible avec <code>get_connection()</code> , et une assertion toujours vraie.	Correcte	Aucune ; exécution confirmée : les 19 tests du projet passent (<code>pytest tests/ -v</code>), dont les 9 tests de <code>storage.py</code> une fois corrigés. La version brute de Gemini, non corrigée, est

#	Étape	Outil	Prompt	Qualité	Correction apportée
			Corrige-les et crée un fichier tests/test_storage.py fonctionnel. »		conservée telle quelle dans backend/ test_storage_gemini.py à titre d'illustration du bug détecté.
9	Correction du dashboard (Chart.js non affiché)	Claude Sonnet 5	« Les graphiques Chart.js ne s'affichent pas dans le dashboard alors que l'API répond correctement (vérifié dans l'onglet réseau du navigateur). Diagnostique et corrige le problème. »	Partielle	J'ai identifié que le CDN externe utilisé pour Chart.js était bloqué par mon réseau, ce qui provoquait un message d'erreur trompeur ("API injoignable" alors que l'API fonctionnait). J'ai fait héberger Chart.js localement dans le projet (frontend/vendor/chart.js) pour ne plus dépendre d'un CDN, et fait séparer la gestion d'erreurs pour distinguer un problème d'API d'un problème de rendu.

Test de formulations différentes (consigne obligatoire)

Pour ce test, j'ai repris une vraie tâche du projet — le nettoyage des données (`cleaner.py`) — et je l'ai demandée à Gemini sous 4 formulations différentes (français/anglais, vague/précis), afin de comparer les résultats à mon code réel.

Formulation	Outil	Qualité	Effet observé
Français, vague : « fais une fonction qui nettoie des données de qualité de l'air »	Gemini	Partielle	J'ai constaté que l'outil supposait de lui-même un DataFrame pandas et des colonnes <code>PM2.5</code> / <code>N02</code> , alors que mon projet utilise une liste de dicts. Il comble les valeurs manquantes par interpolation linéaire au lieu de les garder à <code>None</code> comme mon projet l'exige. Utilisable comme base de réflexion, mais pas directement adaptable à mon code existant.
Français, précis (mêmes règles que ma fonction <code>clean_all</code>)	Gemini	Partielle	La structure obtenue est très proche de mon vrai <code>cleaner.py</code> (liste de dicts, <code>set()</code> pour dédupliquer, bornes 0-1000). J'ai toutefois remarqué qu'une règle de mon projet, non explicitement redemandée, n'était pas reproduite : supprimer une ligne si toutes ses valeurs de polluants sont invalides.

Formulation	Outil	Qualité	Effet observé
			Gemini garde la ligne tant que le timestamp existe, même si tout le reste est <code>None</code> .
Anglais, vague : « <i>write a function to clean air quality data</i> »	Gemini	Partielle	J'ai remarqué que cette version supposait aussi pandas, mais ajoutait d'elle-même une étape de déduplication que la version française vague n'avait pas proposée — un peu mieux deviné sur le besoin réel, sans que je l'aie demandé. Reste inadapté au format liste de dicts de mon projet.
Anglais, précis (traduction de ma version française précise)	Gemini	Partielle	Résultat quasi identique à ma version française précise (code miroir). J'ai retrouvé la même lacune : pas de suppression des lignes entièrement invalides.
Fichier isolé vs projet complet : j'ai envoyé uniquement <code>storage.py</code> à Gemini, sans le reste du projet (<code>config.py</code> , <code>cleaner.py</code> ...)	Gemini	Partielle (voir étape 8)	J'ai conclu que le manque de contexte sur le reste du projet n'était pas la cause du bug rencontré : le fichier fourni contenait déjà le détail technique qui posait problème (<code>get_connection()</code> ouvrant une connexion par appel). L'erreur venait d'une mauvaise inférence sur le comportement de SQLite en mémoire, pas d'un manque d'information. Ma conclusion : envoyer le fichier ciblé suffit pour générer des tests unitaires ; envoyer tout le projet n'aurait probablement pas évité ce bug précis.

Ma conclusion sur ce test : une fois mon prompt rendu précis, la langue (français/anglais) ne change presque rien à la qualité — c'est la précision de ma consigne qui fait la différence, pas la langue. En version vague en revanche, j'ai observé que l'anglais devinait légèrement mieux mon intention (ajout spontané de la déduplication), mais les deux versions vagues se trompaient sur un point structurant : elles supposaient pandas alors que mon projet utilise des dictionnaires natifs. Dans mes deux versions précises, une règle métier que je n'avais pas explicitement redemandée (suppression des lignes totalement invalides) n'a jamais été reproduite spontanément — j'en déduis qu'un prompt précis ne récupère que ce qui est explicitement spécifié, jamais les règles implicites déjà présentes ailleurs dans mon code.

Bilan

Je considère avoir couvert toutes les consignes obligatoires du journal : - j'ai utilisé deux outils IA distincts et documenté chaque usage (Claude pour l'essentiel du projet, Gemini pour les tests de `storage.py` et le test de formulations sur `cleaner.py`) ; - j'ai détecté, expliqué et corrigé un cas concret de code à corriger (les bugs des tests Gemini sur `storage.py`) ; - j'ai mené un test de formulations différentes (français/anglais, vague/précis) sur une vraie tâche de mon projet, avec effet observé et commenté.